FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Carrera: Lic. en Ciberdefensa

Materia
Sistemas Operativos II

Unidad 2 / Semana 5

Año: 2025

Contenidos

Sincronización de procesos. Condiciones de carrera. Región crítica. Semáforos. Mutexes. Monitores. Mensajes entre procesos.

Índice

1. Comunicación entre procesos y Sincronización de procesos 3
1. 1 Condición de Carrera 3
1.2 Región crítica 4
2. Exclusión Mutua 4
2.1 Exclusión Mutua con espera ocupada 5
2.2 Exclusión Mutua con soporte de hardware (TSL) 6
2.3 Exclusión Mutua sin espera ocupada 6
2.3.1 Función sleep / wake up 7
2.4 Operaciones atómicas 7
2.5 Semáforos y mutexes 8
2.6 Otros mecanismos de Sincronización 10
3. El problema del productor-consumidor 11
3.1 Interferencia (Condición de Carrera) 12
4. Implementación de semáforos y mutex en POSIX y Windows 13

1. Comunicación entre procesos y Sincronización de procesos

Hemos visto que en los sistemas operativos modernos, la concurrencia es una característica esencial que permite ejecutar múltiples procesos o hilos de manera simultánea. Esta capacidad, si bien potencia el rendimiento y la eficiencia del sistema, introduce desafíos complejos relacionados con el acceso compartido a recursos, la coordinación entre procesos y la integridad de los datos.

Podemos decir que la IPC se ocupa de tres aspectos en la comunicación entre procesos:

- **Pasaje de información.**
  Esto incluye los diferentes mecanismos que los sistemas operativos tienen para la comunicación entre procesos ya vistos como tuberías, memoria compartida, pasaje de mensajes entre otros.

- **Asegurar que los procesos no se interpongan entre sí**
  Este plano se refiere al momento de competir por recursos compartidos (datos, variables) que deben ser modificados, se haga de forma controlada para evitar resultados inesperados.

- **Ejecutar a los procesos en el orden adecuado según su dependencia**
  Se refiere a que los procesos que guardan cierta dependencia se ejecuten en la secuencia adecuada. Ejemplo: caso productor / consumidor, el proceso consumidor deberá esperar que el productor genere los datos necesarios para que luego el proceso consumidor los comience a utilizar.

El pasaje de información ya fue explicado anteriormente, por lo tanto nos centraremos en los dos restantes que conforman la *Sincronización de procesos*.

Si bien mencionamos *procesos,* esto aplica de igual manera para hilos de forma análoga, dado que el problema es similar y aplican las soluciones aquí propuestas.

1. 1 Condición de Carrera

Llamaremos ***condición de carrera*** a la situación en la que varios procesos o hilos pueden acceder y manipular los mismos datos al mismo tiempo, es decir, de forma concurrente, y donde el resultado de la ejecución depende del orden particular en el que tienen lugar dichos accesos.

Estas situaciones ocurren frecuentemente en los sistemas operativos, puesto que diferentes componentes del mismo manipulan los mismos recursos interfiriendo unos con otros.

1.2 Región crítica

Por otra parte, la ***sección o región crítica*** es la parte del *programa* en que se accede a los recursos compartidos como áreas de memoria, variables globales, archivos entre otros para modificarlos.

Podemos decir entonces que la *condición de carrera* se da cuando dos o más procesos o hilos se encuentran en su *región crítica* al mismo tiempo.

Esta es la situación que hay que evitar y para eso se usa el mecanismo de Exclusión Mutua.

2. Exclusión Mutua

Las condiciones de carrera surgen cuando múltiples procesos acceden simultáneamente a recursos compartidos —como memoria, archivos o dispositivos— y el resultado final depende del orden en que se ejecutan. Para evitar este tipo de problemas, se deben implementar mecanismos que impidan que más de un proceso lea o modifique los mismos datos al mismo tiempo.

Este principio se conoce como ***exclusión mutua***: una técnica que garantiza que, si un proceso está utilizando una variable o archivo compartido, los demás procesos deben quedar temporalmente excluidos de ese acceso.
En el ejemplo clásico, el error ocurre cuando el proceso B comienza a usar una variable compartida antes de que el proceso A haya terminado de modificarla, generando resultados inconsistentes.

Existen diferentes formas de implementar la exclusión mutua, algunas basadas en hardware, otras en software o funciones disponibles en el sistema operativo (primitivas).

Sin embargo, garantizar la exclusión mutua no es suficiente por sí sola. Para que los procesos concurrentes cooperen de manera correcta y eficiente, es necesario que cualquier solución cumpla con los siguientes requisitos:

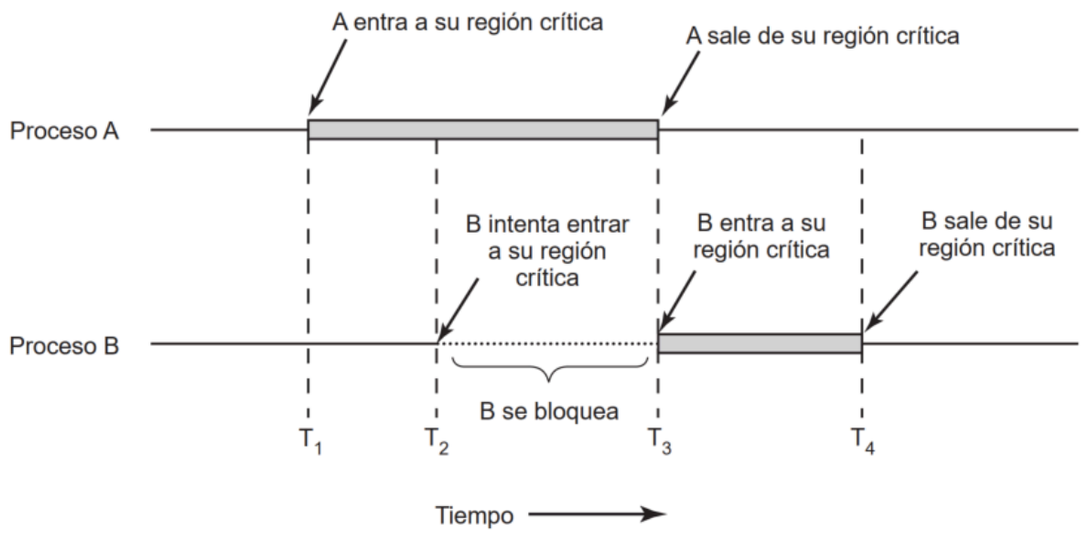
- Exclusión: No debe haber dos procesos simultáneamente en sus regiones críticas.

- Independencia del hardware: La solución no debe depender de la velocidad relativa de los procesos ni del número de CPUs.

- No interferencia externa: Un proceso que no esté en su región crítica no debe bloquear a otros.

- Espera limitada: Ningún proceso debe quedar esperando indefinidamente para entrar a su región crítica.

Estos criterios constituyen la base para evaluar la calidad de cualquier mecanismo de sincronización en sistemas operativos.



Exclusión mutua mediante el uso de regiones críticas[1]

2.1 Exclusión Mutua con espera ocupada

La espera ocupada (busy waiting) ocurre cuando un proceso no puede entrar a la región crítica y, en lugar de ceder el procesador, se queda en un bucle revisando constantemente si ya puede entrar.
Esto consume la CPU innecesariamente, no es una solución óptima pero es útil para entender los primeros enfoques de sincronización.
Un ejemplo clásico es la solución de Peterson que usa bucles activos para evaluar si se cumple la condición de ingresar a la región crítica.

```
bool flag[2];    // indica si el proceso quiere entrar
```

[1] Tanenbaum, Andrew (2009), Sistemas Operativos Modernos. Tercera edición

```
int turn;        // indica a quién le toca

// Código para proceso Pi

flag[i] = true;

turn = j;

while (flag[j] && turn == j);  // espera ocupada

// región crítica

...

flag[i] = false;  // salida
```

2.2 Exclusión Mutua con soporte de hardware (TSL)

Otras implementaciones de la exclusión mutua utilizan una variable llamada candado para controlar el acceso a la región crítica. Y una mejora de esta técnica es usar ayuda del hardware para almacenar y actualizar el contenido de la variable candado especialmente en ambientes con más de un procesador.

La instrucción TSL (Test and Set Lock) es una operación atómica usada en sistemas multiprocesador para implementar exclusión mutua.

Su función es:

- Leer el valor de una variable compartida llamada candado.

- Guardarlo en un registro.

- Escribir un valor distinto de cero en la variable candado.

Estas acciones se realizan de forma indivisible, sin interrupciones, bloqueando el acceso de otros procesadores a esa memoria mientras dura la instrucción.

Para desbloquearlo simplemente se almacena el valor cero en el candado.

2.3 Exclusión Mutua sin espera ocupada

Tanto la solución de Peterson como las implementaciones basadas en el uso de variables candado o en instrucciones como TSL son correctas desde el punto de vista de la exclusión mutua.

Sin embargo, todas comparten una limitación importante: ***requieren espera ocupada***. En esencia, estas técnicas verifican si el acceso a la región crítica está

permitido en el momento en que un proceso intenta ingresar. Si no lo está, el proceso permanece en un bucle de espera activa, consumiendo ciclos de CPU hasta que se le permita el ingreso.

Este enfoque no solo resulta ineficiente, sino que puede generar efectos indeseados. como sucede en la llamada *Inversión de Prioridades*[2].

Para evitar este tipo de problemas, se usa la *exclusión mutua sin espera ocupada,* a través de diferentes mecanismos que veremos a continuación:

- Funciones predefinidas o primitivas
- Semáforos / Mutex
- Barreras y Monitores

2.3.1 Función sleep / wake up

Estas funciones implementadas en el SO bloquean al proceso en lugar de hacerlo esperar activamente. Una de las más simples es el par de llamadas al sistema sleep (dormir) y wakeup (despertar).

- sleep suspende al proceso que la invoca, colocándolo en estado bloqueado hasta que otro proceso lo reactive.

- wakeup se utiliza para reactivar un proceso específico, indicado como parámetro.

En algunas variantes, ambas llamadas pueden incluir una dirección de memoria como parámetro, que sirve para asociar el evento de suspensión con el de reactivación, permitiendo una coordinación más precisa entre procesos.

Antes de avanzar con el análisis de los mecanismos de sincronización restantes, es necesario introducir un concepto clave, esencial para comprender cómo funcionan y se articulan correctamente estos métodos.

Este concepto constituye la base de la sincronización entre procesos y es determinante para garantizar su correcta coordinación y exclusión mutua..

2.4 Operaciones atómicas

Una operación atómica es una **acción o conjunto de acciones** que se ejecutan completamente o no se ejecuta en absoluto, sin posibilidad de que otro proceso la interrumpa o vea un estado intermedio.

En otras palabras, **es indivisible**: no puede ser dividida ni interferida por otros procesos o hilos mientras se está ejecutando.

[2] Es un problema que ocurre en sistemas operativos con planificación por prioridades, cuando un proceso de *baja prioridad* bloquea un recurso que necesita uno de *alta prioridad*, y este último no puede avanzar porque el primero no se ejecuta por su baja prioridad.
Tanenbaum, Andrew (2009), Sistemas Operativos Modernos. Tercera edición

Veremos la importancia de la implementación de este concepto en el manejo y funcionamiento de semáforos y mutex entre otros mecanismos de sincronización.

2.5 Semáforos y mutexes

Los semáforos son una herramienta de **sincronización** que ofrece una solución al problema de la sección crítica (porción de código de un programa de computador en la cual se accede a un recurso compartido que no debe ser accedido por más de un proceso o hilo en ejecución).

Un semáforo provee una simple pero útil abstracción para controlar el acceso de múltiples procesos a un recurso común.

El concepto de semáforo fue desarrollado por el holandés Esdger W. Dijkstra en 1965. Dijkstra estaba involucrado en el diseño de un sistema operativo como una colección de procesos secuenciales cooperantes y con el desarrollo de mecanismos eficientes y fiables para dar soporte a la cooperación. Estos mecanismos podrían ser usados fácilmente por los procesos de usuario si el procesador y el sistema operativo colaborarán en hacerlos disponibles[3].

El principio fundamental es éste: dos o más procesos pueden cooperar por medio de simples señales (estados de variables), tales que un proceso pueda ser suspendido hasta que haya recibido una señal específica que lo active.

Cualquier requisito complejo de coordinación puede ser satisfecho con la estructura de señales apropiada.

Los semáforos son variables de tipo de dato entero que son controlados por el sistema operativo. Tienen tres acciones posibles:

1. **Inicialización**
   Se pueden inicializar a cualquier valor entero que representa la cantidad de recursos compartidos a sincronizar. Una vez inicializado (por única vez), no es posible leer su valor, dado que son estructuras abstractas y no comparten su valor.
2. **Decrementar**
   Cuando un hilo *decrementa* el semáforo, **si el valor es negativo, el hilo se bloquea** y no puede continuar hasta que otro hilo incremente el semáforo.

[3] Tanenbaum, Andrew (2009), Sistemas Operativos Modernos. Tercera edición

Según la implementación, esta operación puede denominarse **wait, down, acquire** o incluso **P** (por ser la inicial de *proberen te verlagen,* "intentar decrementar" en holandés, del planteamiento original en el artículo de Dijkstra)

3. **Incrementar**
   Cuando un hilo incrementa el semáforo, si hay hilos esperando, uno de ellos se activa (despierta). Los nombres que recibe esta operación son **signal**, **up**, **release**, **post** o **V** (de *verhogen*, "incrementar" en holandés)

La actualización de los semáforos es crítica porque depende de ello que la sincronización de procesos funcione correctamente.

Por ese motivo, todas las operaciones con semáforos o sus primitivas (implementación de las acciones por el sistema operativo) se realizan como "**operaciones atómicas**" que hemos abordado previamente.

Semáforos binarios (mutex)

Los semáforos binarios son un caso especial de semáforos pero ampliamente utilizados. Dijkstra dio en 1968 una solución elegante y sencilla al problema de la exclusión mutua con la introducción del concepto de semáforo binario. Esta técnica permite resolver la mayoría de los problemas de sincronización entre procesos y forma parte del diseño de muchos sistemas operativos y de lenguajes de programación concurrentes.

El nombre **mutex** proviene de "mutual exclusion" que resuelve el problema de sincronización.

Un semáforo binario es un indicador de condición (S) que registra si un recurso está disponible o no. Un **semáforo binario sólo puede tomar dos valores: O y 1**. Si, para un semáforo binario. **S=1** entonces el recurso **está disponible** y la tarea lo puede utilizar; si **S=0** el recurso **no está disponible** y el proceso debe esperar.

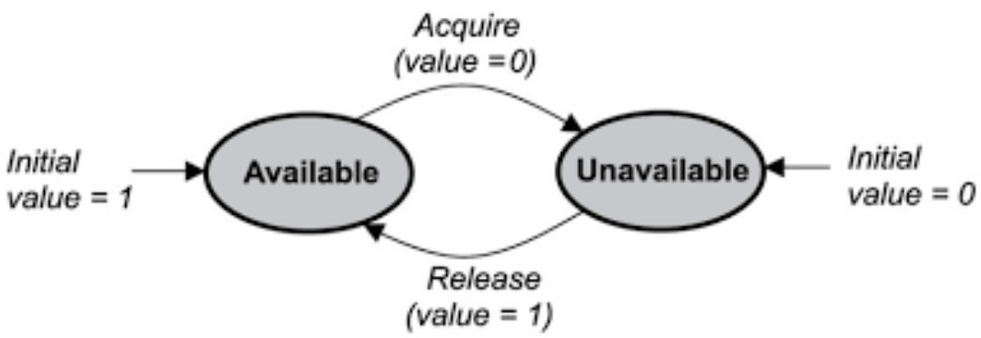


Diagrama de estados de un semáforo binario

Variables de condición

Una variable de condición es un mecanismo de sincronización que permite a un hilo esperar hasta que otro hilo le notifique que una determinada condición se ha cumplido. Se utilizan junto con mutexes (semáforos binarios) para coordinar el acceso a recursos compartidos.

2.6 Otros mecanismos de Sincronización

Monitores

Los monitores son estructuras provistas por el *lenguaje de programación* o entorno de desarrollo que encapsulan tanto los datos como las funciones que los pueden manipular, impidiendo el acceso directo a las funciones potencialmente peligrosas. En otras palabras, son tipos de datos abstractos (Abstract Data Types, ADT), clases de objetos, y exponen una serie de métodos públicos, además de poseer métodos privados que emplean internamente.

Al no exponer al usuario o programador una interfaz susceptible de ser alterada, el monitor encapsula todo el código necesario para garantizar el acceso concurrente seguro a los datos, centralizando su control en un único bloque estructurado.

Un monitor puede implementarse utilizando cualquiera de los mecanismos de sincronización presentados anteriormente —la diferencia radica en que esto

se hace en un solo lugar. Los programas que quieran emplear el recurso protegido lo hacen incluyendo el código del monitor como módulo/biblioteca, lo cual fomenta la reutilización de código.

Como ejemplo, el lenguaje de programación Java implementa sincronización vía monitores entre hilos como una propiedad de la declaración de método, y lo hace directamente en la JVM.(synchronized)

Barreras

Algunas aplicaciones se dividen en fases y tienen la regla de que ningún proceso puede continuar a la siguiente fase sino hasta que todos los procesos estén listos para hacerlo. Para lograr este comportamiento, se coloca una barrera al final de cada fase. Cuando un proceso llega a la barrera, se bloquea hasta que todos los procesos han llegado a ella.
La utilización de una barrera se ilustra en la figura

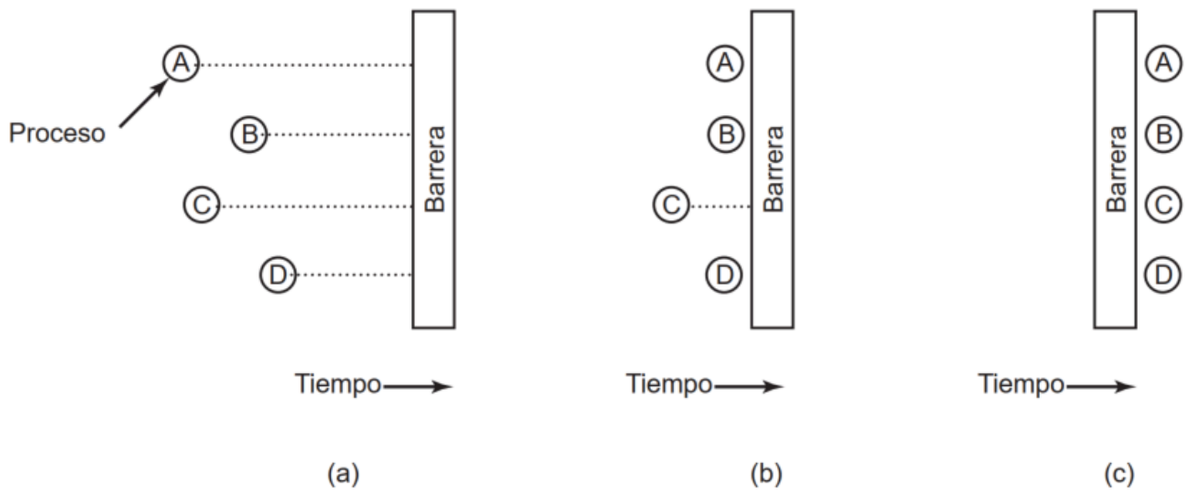


Figura 2-37. Uso de una barrera. (a) Procesos acercándose a una barrera. (b) Todos los procesos, excepto uno, bloqueados en la barrera. (c) Cuando el último proceso llega a la barrera, todos se dejan pasar.

Tanenbaum, Andrew (2009), Sistemas Operativos Modernos. Tercera edición

Pasaje de mensajes

El pasaje de mensajes es un mecanismo de sincronización y comunicación entre procesos que se utiliza especialmente cuando no hay memoria compartida o se desea evitar su uso. Es muy utilizado en sistemas distribuidos, microkernels y entornos donde los procesos deben mantenerse aislados.

Ya se ha descrito su funcionamiento anteriormente.

3. El problema del productor-consumidor

Imagina dos procesos: un **productor** que genera datos (como un proceso que lee datos de un archivo) y un **consumidor** que procesa esos datos (como un proceso que guarda esos datos en una base de datos). Para que estos procesos cooperen,

necesitan compartir un espacio de memoria intermedio, como un **búfer circular**. El productor llena el búfer con datos, y el consumidor los saca para procesarlos.

- **Proceso Productor:** Escribe datos en el búfer. Si el búfer está lleno, se detiene y espera a que el consumidor consuma datos.
- **Proceso Consumidor:** Lee datos del búfer. Si el búfer está vacío, se detiene y espera a que el productor genere más datos.

La cooperación se logra porque ambos procesos tienen acceso al mismo búfer y se comunican indirectamente a través de su estado (lleno o vacío), asegurando que trabajen de forma coordinada sin sobrescribir o leer datos incorrectos.

3.1 Interferencia (Condición de Carrera)

La interferencia puede observarse claramente en una condición de carrera. Supongamos que dos procesos, P1 y P2, intentan incrementar el valor de una variable compartida llamada contador, cuyo valor inicial es 0. Si ambos procesos acceden simultáneamente a dicha variable sin mecanismos de sincronización adecuados, el resultado final puede ser incorrecto o impredecible, evidenciando el problema de interferencia.

El proceso para incrementar la variable es:

1. Leer el valor actual del contador.
2. Incrementar el valor leído en 1.
3. Escribir el nuevo valor en contador.

Si ambos procesos intentan hacer esto al mismo tiempo sin sincronización, podría ocurrir lo siguiente:

1. **P1** lee contador (valor 0).
2. **P2** lee contador (valor 0) al mismo tiempo.
3. **P1** incrementa su copia a 1 y la escribe en el contador.
4. **P2** incrementa su copia a 1 y la escribe en el contador.

Al final, el valor de contador es **1**, cuando *debería ser **2*** (uno por cada proceso). Ambos procesos interfirieron en la operación del otro al acceder al recurso

compartido sin control, lo que llevó a un resultado erróneo. Los mecanismos de sincronización, como los **semáforos** o **mutexes**, sirven para prevenir este tipo de interferencia al asegurar que solo un proceso a la vez acceda a la sección crítica (el código que modifica la variable).

Un **mutex** (abreviatura de *mutual exclusión*) es una de las herramientas de sincronización más comunes que se usan para prevenir este problema como ya hemos visto.

Ejemplo de cómo un mutex evita la interferencia

Tomemos el ejemplo anterior de los procesos P1 y P2 que incrementan una variable contador de forma concurrente. Para evitar la condición de carrera, podemos proteger la sección crítica con un mutex.

1. Se inicializa un **mutex**, que puede estar en dos estados: **bloqueado** (locked) o **desbloqueado** (unlocked). Inicialmente, está desbloqueado.
2. Antes de que un proceso entre en la sección crítica, debe intentar **bloquear** el mutex.
3. Después de salir de la sección crítica, debe **desbloquear** el mutex.

Esta solución se suele complementar con la utilización de una **variable de estado** para controlar la situación de **buffer vacío** para evitar que el consumidor intente leer datos o de **buffer lleno** para controlar el caso que, con el buffer lleno el producto siga intentando grabar datos.

4. Implementación de semáforos y mutex en POSIX y Windows

En los sistemas operativos modernos, especialmente aquellos compatibles con el estándar **POSIX** (Portable Operating System Interface), la sincronización entre procesos e hilos es muy importante para garantizar la consistencia de datos y evitar condiciones de carrera.
Cuando múltiples hilos o procesos acceden simultáneamente a recursos compartidos, es necesario coordinar ese acceso para evitar interferencias y resultados erróneos.

Dos de las herramientas más utilizadas para este propósito son los mutexes y los semáforos, ambos definidos por las bibliotecas POSIX. Aunque comparten el objetivo de controlar el acceso a secciones críticas, tienen diferencias clave en su comportamiento y aplicabilidad.
También Windows API soportan semáforos y ambos admiten dos tipos de semáforos:

- Los **semáforos anónimos** que solo existen en el espacio de direcciones del proceso que los crea, de tal forma que están disponibles para sincronizar hilos del mismo proceso. La forma de usarlos para sincronizar procesos diferentes o hilos en procesos diferentes depende del sistema operativo. Con *Windows API* se pueden heredar de padres a hijos. Mientras que en sistemas POSIX es necesario crear el semáforo en una región de memoria compartida, que hayamos creado previamente, e indicar un valor distinto de 0 en el argumento pshared de sem_init().

- Los **semáforos con nombre** son públicos al resto del sistema, por lo que teóricamente cualquier proceso con permisos puede abrirlos para utilizarlos.

	POSIX API	Windows API
Crear semáforo anónimo	sem_init()	CreateSemaphore()
Crear semáforo con nombre	sem_open()	CreateSemaphore()
Abrir semáforo con nombre	sem_open()	OpenSemaphore()
Operación acquire	sem_wait()	WaitForSingleObject()
Operación release	sem_post()	ReleaseSemaphore()
Cerrar semáforo con nombre	sem_close()	CloseHandle()
Destruir semáforo anónimo	sem_destroy()	[Automático]
Destruir semáforo con nombre	sem_unlink()	[Automático]

Funciones de la API para manipular semáforos[4].

[4] Torres, Jesús: Sistemas Operativos edición OCW. Disponible en línea: (https://ull-esit-sistemas-operativos.github.io/ssoo-apuntes/ocw/main.html